
Arduino ～計画書と設計書～ オーケストラ

目次

第 1 章	システムの基本設計	1
1.1	機能一覧	1
1.1.1	機能分解 (FBS)	1
1.1.2	機能と性能指標 (MOP) の対応	2
1.1.3	各 MOP 目標値の根拠	2
1.2	システム構成図	4
1.2.1	全体ブロック図	4
1.2.2	ノード役割分担	5
1.2.3	データフロー	6
1.2.4	採用アーキテクチャ (EMA) の方針	6
1.3	操作インターフェース設計	6
1.3.1	ユーザー操作	7
1.3.2	LED 表示仕様	7
1.3.3	通信プロトコル方針 (CTRL / BEAT / NOTE)	7
1.3.4	通信方式 (WiFi UDP + USB Serial の二系統)	8
1.3.5	パケロス・ジッタ対策	9
1.4	状態遷移図	9
1.4.1	指揮者ノード (node_01)	10
1.4.2	楽器ノード (node_02~05)	10
第 2 章	システムの詳細設計	13
2.1	部品一覧	13
2.2	ハードウェア設計	15
2.2.1	配線・ピン	15
2.2.2	ネットワーク構成	16
2.2.3	物理配置	16
2.3	ソフトウェア設計	17
2.3.1	ファイル構成	17
2.3.2	オブジェクト設計 (クラス図)	18
2.3.3	関数設計	19
2.4	処理フローの設計	27
2.4.1	共通: 3 フェーズ実行モデル	27

2.4.2	指揮者ノード (node_01)	27
2.4.3	楽器ノード (node_02-05)	34
2.5	テストの設計	42
2.5.1	試験戦略 (単体 → 結合 → システム)	42
2.5.2	試験項目 (TPM)	43
2.5.3	要求 → 検証の対応	44
2.5.4	テスト実装で確定した値 (Version 0.2 追記)	45
2.5.5	実機検証で確定する未確定事項	47
第 3 章	生成 AI の利用について	49
第 4 章	参考文献	51

第 1 章 システムの基本設計

本章では、「Arduino オーケストラ」(指揮者ノード 1 + 楽器ノード 4 の計 5 ノード)のうち塩澤担当範囲(指揮者ノード node_01 と楽器ノード node_02~node_05 のファームウェア)を対象に、システム全体の機能・構成・操作インターフェース・状態遷移を基本設計レベルでまとめる。楽器構成は **金管 3 + ドラム 1** とし、各楽器ノードは**専用の Mac (Processing 実行) と 1 対 1 で USB 接続**される。Mac 側の音色合成・スピーカ出力(梅澤担当)は本章のスコープ外で、データ受け渡し境界(NOTE パケット)のみを定義する。通信は **ノード間(指揮者 → 楽器) = WiFi UDP マルチキャスト**, **Arduino-Mac 間(楽器 → 各 Mac) = USB Serial (UART, Serial.write)** の二系統に分離した構成を採る。

ファームウェアの設計原則は、外部 OSS であり著者本人が公開している **Embedded-Module-Architecture (以下 EMA)**¹⁾ に全面準拠する。EMA の中核ルール (IModule インターフェース, 3 フェーズループ, SystemData と ProjectConfig の集約) は §1.2 で位置づけを示し、第 2 章で具体化する。

1.1 機能一覧

ファームウェア全体を機能分解構造(FBS)で整理する。本書スコープは Arduino 系のみであり、楽器音色や発表準備などチーム全体側の要素は含めない。

1.1.1 機能分解 (FBS)

- F1. 指揮情報抽出 (node_01)** F1.1 IMU 6 軸の取得, F1.2 信号前処理 (IIR LPF + ノルム), F1.3 拍検出 (振り下ろしピーク), F1.4 テンポ推定 (拍間隔 → BPM), F1.5 強弱推定 (振幅 → velocity, ストレッチ), F1.6 演奏状態管理。
- F2. 指揮情報配信 (node_01)** F2.1 CTRL 送出 (BPM・velocity・state), F2.2 BEAT 送出 (拍トリガ), F2.3 SoftAP 起動・維持 (自身がアクセスポイントを兼ね, 楽器ノードを収容)。
- F3. 楽譜進行 (node_02~05)** F3.1 CTRL/BEAT 受信, F3.2 楽譜データ保持 (C 配列・パート別), F3.3 BEAT 同期での音符進行, F3.4 NOTE イベント生成, F3.5 パート固有ロジック (partId・startBeatNo)。
- F4. 発音情報配信 (node_02~05 → PC)** F4.1 NOTE パケット生成, F4.2 Processing への USB Serial 送信 (Serial.write(), CDC 1:1 接続)。

1) <https://github.com/takushio2525/Embedded-Module-Architecture>

F5. 共通基盤（全ノード） F5.1 IModule 登録・3 フェーズループ, F5.2 ModuleTimer による周期実行, F5.3 SystemData/ProjectConfig 集約, F5.4 起動シーケンス・診断 LED, F5.5 init 失敗リトライ・パケロス時のフォールバック.

ストレッチ機能（F1.5 強弱推定）は必達機能の実装完了後に着手する. 詳細設計ではインターフェースだけ用意し, 初期実装ではスタブで通す構成にする.

1.1.2 機能と性能指標（MOP）の対応

各機能に対する性能指標を表 1.1 に示す. 目標値は原案段階のたたき台であり, 実機計測後に第 2.5 節で再調整する.

表 1.1: 機能と性能指標の対応

ID	指標	目標値
MOP-1	楽器間 同期誤差（4 ノードの発音タイミング差）	$\leq 20 \text{ ms}$
MOP-2	指揮 → 楽器 通信遅延（送信～受信）	$\leq 10 \text{ ms}$
MOP-3	拍検出 正解率（定速指揮時）	$\geq 90 \%$
MOP-4	テンポ追従 遅延（BPM 階段変化への追従）	$\leq 2 \text{ 拍}$
MOP-5	パケロス耐性（聴感破綻なし）	5 % パケロス
MOP-6	入力フェーズ CPU 時間（1 周期）	$\leq 2 \text{ ms}$
MOP-7	起動時間（電源投入 → 演奏可能）	$\leq 5 \text{ s}$
MOP-8	強弱制御 分解能（ストレッチ）	3 段階以上 (p/mf/f)

1.1.3 各 MOP 目標値の根拠

各 MOP の数値は, 演奏体験として「合っている／追従している」と感じられる水準と, 採用ハードウェア（XIAO ESP32-S3 Sense + MPU6050, SoftAP+UDP マルチキャスト）で達成可能と見込まれる水準の, 両者が成立する範囲で設定した. 以下に MOP ごとの根拠を示す.

MOP-1（楽器間 同期誤差 $\leq 20 \text{ ms}$ ） 合奏における演奏者間タイミング調整の心理学的研究をまとめた河瀬 [1] は, Rasch (1979) や Keller & Appel (2010) などの先行研究を引いて, 「演奏者どうしのズレが**数 10 ms 程度ならば, 演奏者・聴き手ともズレているとは感じない**」一方で **100 ms を超えると合っていないと判断される**ことを整理している. 本システムでは ADR-0006 に基づき, 知覚的に 2 音が分離して聴こえ始める

目安として広く用いられる **20 ms** を目標値に採る。本システムは同一 SoftAP 配下の LAN 内通信で物理距離由来の遅延は無視できるため、マスタクロック方式 (§2.3.3.6) と BEAT 連送 (beatRedundancy) の組み合わせでこの水準を狙う。

MOP-2 (通信遅延 ≤ 10 ms) 本システムでは BEAT 受信後に楽器ノードが playAtMasterMs まで保持して発音するマスタクロック方式 (§2.3.3.6) を採るため、通信遅延そのものは MOP-1 (同期誤差) にほぼ吸収される。したがって MOP-2 は MOP-1 の許容範囲 (20 ms) の半分程度を割り当てた**設計マージン**として 10 ms を採る。広帯域ネットワークを用いた遠隔同時音楽演奏の実験 [2] でも、演奏が破綻しない通信遅延の上限は楽曲・テンポ依存ながら十数 ms ~ 数十 ms オーダーに収まることが報告されており、ESP32 + UDP マルチキャストによる同 LAN 内通信は十分この範囲に入る。

MOP-3 (拍検出 正解率 ≥ 90 %) 加速度ベースの指揮ジェスチャ認識は、HMM とファジィ推論を組み合わせたマルチモーダル指揮シミュレータの先行研究 [3] で既に実用域の認識精度が示されている。新山王 [4] による指揮基本動作の分析でも、熟達者と初心者で打点の特徴差が定量化されており、**加速度ノルム閾値による振り下ろし検出は実装容易性の高い手法**として裏づけられる。本ハッカソンでは演奏破綻を起こさない実用水準として、IR/分類器評価で「実用域」とされる 90 % を目標にした。

MOP-4 (テンポ追従 ≤ 2 拍) 指揮基本動作の分析 [4] が示すとおり、指揮の打点と拍点の関係から人間の指揮者・演奏者は **1-2 拍以内でテンポ変化を追従**する。本システムはこの「人間と同等の追従感」を最低水準とし、BPM 推定に EMA (指数移動平均) を用いるため、2 拍以内で目標 BPM の許容誤差に収まる設計とした。

MOP-5 (5 % パケロス下で聴感破綻なし) ITU-T 勧告 Y.1541 の日本語版である TTC 標準 JT-Y1541 [5] は、IP ベースサービスの品質クラスごとにパケットロス率の目標値を規定しており、最も厳しいクラス 0 (リアルタイム高品質) でロス率 1×10^{-3} (0.1 %), 順次クラス 1 / 2 / 3 で同値、クラス 4 で 1×10^{-3} , クラス 5 (規格外・通常通信) では未規定としている。家庭用 IP 通信の実観測ベースの解説 [6] でもロス率 1 % 以下が良好、3 % 超で不具合が顕在化、10 % 超でストレスを感じる水準とまとめられており、無線 LAN ではマルチキャストの取りこぼしが有線より有意に多い [7]。本システムは BEAT 連送 (beatRedundancy = 2) と「過去 BEAT も即発火する寛容な受信側」 (§2.3.3.6) でパケロスを吸収する設計であり、試験条件としては「規格目標を 1 桁以上上回る悪化条件でも演奏破綻しないこと」を確認する意味で 5 % を採用する。Version 0.1 で想定していた SelfRun 自走は、テスト実装で「指揮者再起動中に楽器だけ走る」等の副作用が出たため 0.2 で撤廃した。

MOP-6 (入力フェーズ CPU 時間 ≤ 2 ms) MPU6050 の I2C サンプリングを 200 Hz (周期 5 ms) で行う設計のため、EMA の 3 フェーズ実行モデル (入力 $\rightarrow$ applyPattern() $\rightarrow$ 出力) の各フェーズ平均が $5/3 \approx 1.7$ ms 以内に収まる必要がある。入力フェーズに

2 ms 以下の制約を置けば、残り 3 ms をロジックと出力に割り当てて 5 ms 周期を破綻なく回せる。数値は文献ではなく、サンプリング周波数とループ周期から導出した**実時間制約**である。

MOP-7 (起動時間 ≤ 5 s) この値は外部文献ではなく**設計上の見積もりとユーザー体験の経験則**から定めた。ESP32 系における SoftAP 起動・STA 接続・初期 DHCP 等で 2-3 秒、指揮者ノードの Calibrating (静止ノルム学習) に 2 秒を要する想定で、合計 5 秒以内に最初の CTRL を送出する。実演前のセットアップ時間としてユーザーが「いつ振り始めてよいか」を判断できる範囲はおおよそ 5 秒以内であり、体験面の閾値とも整合する。本値は §2.5.5 (実機で確定する仮値) に従い実測で再調整する。

MOP-8 (強弱 3 段階以上) 西洋音楽の強弱記号体系では **p (ピアノ=弱く) / mf (メゾフォルテ=やや強く) / f (フォルテ=強く)** が広く用いられる基本単位であり (楽典の周知事項)、3 段階あれば多くの初級～中級曲のダイナミクス指示を最低限再現できる。ストレッチ機能 (F1.5) の初期実装としてこの 3 段階分離を達成水準と置く。

1.2 システム構成図

1.2.1 全体ブロック図

5 ノードと Mac × 4 の関係を図 1.1 に示す。指揮者 **node_01** 自身が **SoftAP (ESP32-S3 のソフトウェア AP)** を立て、楽器 **node_02~05** は STA としてこれに接続する。外部ルータやスマホテザリングは不要。node_01 は CTRL/BEAT を **UDP マルチキャスト** で送信し、node_02~05 はそれぞれグループに join して受信する。各楽器は楽譜を進行させながら NOTE を **USB Serial (Serial.write)** で**専用 Mac (1 対 1 接続)** へ送信する。Arduino-Mac 間は WiFi を介さず、給電と Serial 通信を兼ねた USB 接続で直結する。

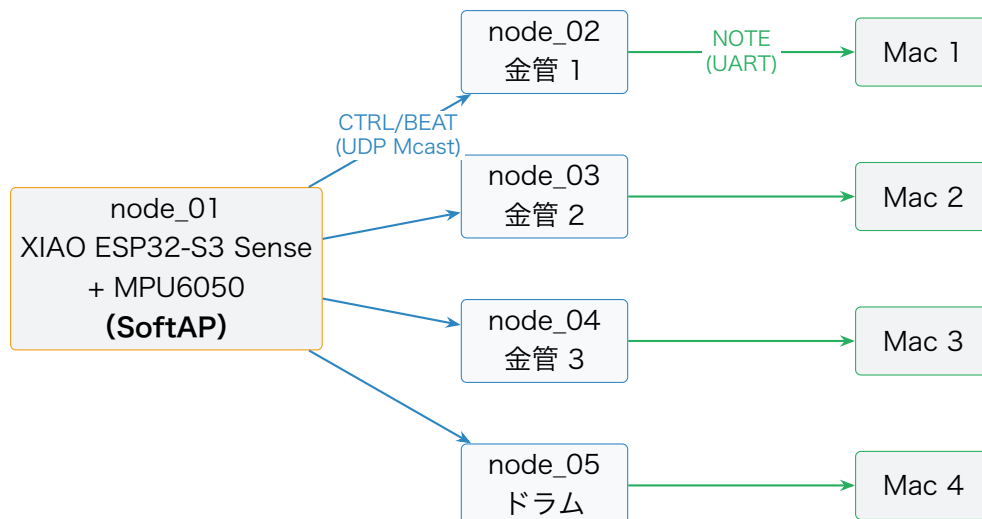


図 1.1 全体ブロック図（指揮者 node_01 が SoftAP を兼ね、楽器 4 台が STA 接続、楽器 → Mac は 1 対 1 USB Serial）

1.2.2 ノード役割分担

各ノードの責務とコード担当を表 1.2 に示す。node_02～05 の Arduino コードは **共通ソース + パート設定差し替え**（ProjectConfig と楽譜データのみ差分）で構成し、4 台分の別コードは書き分けない (§2.4.3)。

表 1.2: ノード役割分担

ノード	物理配置	主な責務	コード担当
node_01	指揮者の手・指揮棒	MPU6050 読み取り → 拍・テンポ・強弱推定 → CTRL/BEAT 送出、 自身が SoftAP を起動 し楽器を収容	塩澤
node_02	金管 1	BEAT 同期で楽譜進行 → NOTE 送出（金管 1 パート）	塩澤
node_03	金管 2	BEAT 同期で楽譜進行 → NOTE 送出（金管 2 パート）	塩澤
node_04	金管 3	BEAT 同期で楽譜進行 → NOTE 送出（金管 3 パート）	塩澤
node_05	ドラム	BEAT 同期で楽譜進行 → NOTE 送出（ドラム）	塩澤
Mac × 4	楽器ごとに 1 台	UART で NOTE を受信 → 倍音・ADSR 調整 → 波形生成 → 内蔵スピーカ出力	梅澤（参考）

1.2.3 データフロー

演奏中の 1 拍ぶんの情報の流れを擬似シーケンスで示す。

1 拍ぶんのデータフロー（演奏中）

- (1) MPU6050 (I2C) → node_01: 加速度・角速度サンプル (200 Hz).
- (2) node_01 内: applyPattern() がフィルタ・拍検出・テンポ推定を実行.
- (3) 振り下ろしを検出: node_01 (SoftAP) → node_02~05 に **BEAT** (beatNo, timestampMs) を UDP マルチキャスト.
- (4) node_02~05 内: 楽譜ポインタ前進 → NoteOn フラグ設定.
- (5) node_02~05 → **各 Mac (1 対 1)**: **NOTE** (noteNumber, velocity, durationMs) を USB Serial (Serial.write) で送信.
- (6) 並行して node_01 は 20 Hz で **CTRL** (BPM・velocity・state) を同じマルチキャストグループへ送信し続ける.

1.2.4 採用アーキテクチャ (EMA) の方針

全 5 ノードのファームウェアは EMA に準拠する。EMA の中核ルールを本ハッカソンに当てはめた要旨を以下に示す。バイト単位のパケット定義や具体コードは第 2 章で示す。

- ・ **IModule インターフェース**: 全ハードウェアモジュールが init() / updateInput() / updateOutput() / deinit() の 4 メソッドを実装する.
- ・ **3 フェーズ実行モデル**: loop() は「入力 → ロジック applyPattern() → 出力」の順で必ず実行する.
- ・ **SystemData 集約**: モジュール間共有状態を一元化する構造体。モジュール同士の直接呼び出しは禁止.
- ・ **ProjectConfig 集約**: 各モジュールの設定を{MODULE}_CONFIG インスタンスとして 1 ヘッダに集約.
- ・ **判断ロジックの位置づけ**: 拍検出・テンポ推定・楽譜進行のような「ハードウェアに触れない判断ロジック」は IModule の派生にせず、applyPattern() 関数内に書く.

1.3 操作インターフェース設計

本節では「操作インターフェース」を、**ユーザーが触れる物理 IF** (指揮棒・LED) と **ノード間/PC 間の通信プロトコル IF** (CTRL/BEAT/NOTE) の二層に分けて定義する。通信は **Arduino-Arduino 間 = WiFi UDP**, **Arduino-PC 間 = USB Serial** の二系統に分け、それぞれ役割と性質が異なる。バイト単位のパケットレイアウトは §2.3.3 で具体化する。

1.3.1 ユーザー操作

- ・ **指揮棒（指揮者）**： MPU6050 を搭載した node_01（XIAO ESP32-S3 Sense）を指揮棒として手に持ち、通常の指揮動作（4 拍子・3 拍子の振り下ろし）で操作する。特別なボタン等は持たない。
- ・ **起動順**： 楽器ノード（node_02～05）を先に起動し（Idle 状態で SoftAP を待機）、その後指揮者ノード（node_01）を起動する。指揮者の SoftAP 起動で楽器が WaitStart に遷移し、指揮者の Conducting 入りで BEAT が流れ始めて楽器が Playing に遷移する。
- ・ **Calibrating**： 指揮者ノードは起動後、最初の 2 秒間に静止時の加速度ノルムを平均し、静止ノルム（ $\approx$ 重力 1 g）として記録する。使用者は単に「起動後 2 秒間、腕を自然に下ろした状態で待つ」だけでよい。

1.3.2 LED 表示仕様

各ノードは内蔵 LED（LED_BUILTIN）の点滅パターンで状態を提示する（表 1.3）。点滅周期は StatusLedConfig.blinkIntervalMs で調整可能。

表 1.3: LED 表示パターン

ノード	状態	LED パターン	意味
node_01	Idle	低速点滅（1 Hz）	起動直後・SoftAP 起動前
node_01	Calibrating	中速点滅（2 Hz）	静止ノルム（ $\approx$ 1 g）学習中（2 秒）
node_01	Conducting	点灯	通常演奏中
node_01	Fallback	高速点滅（5 Hz）	IMU エラー or SoftAP 異常
node_02-05	Idle	低速点滅（1 Hz）	起動直後・SoftAP 未接続
node_02-05	WaitStart	中速点滅（2 Hz）	SoftAP 接続済、最初の BEAT 待ち
node_02-05	Playing	点灯	通常演奏中（次 BEAT が長く来なくても Playing に留まる）

1.3.3 通信プロトコル方針（CTRL / BEAT / NOTE）

3 種類のメッセージを使い分ける（表 1.4）。

表 1.4: 3 種類のメッセージの設計方針

種別	送信元 → 宛先	送信契機	性質	役割
CTRL	node_01 node_02-05 Mcast)	→ 定期 (20 Hz) (UDP	冪等	BPM ・ velocity ・ state を継続同期
BEAT	node_01 node_02-05 Mcast)	→ 拍 検 出 時 (不 (UDP 定期)	即時性重視	楽譜ポインタ前進ト リガ
NOTE	node_02-05 → 各 Mac (USB Serial, 1:1)	楽 譜 イ ベ ン ト 発火時	順序保証	自パート Mac への 発音依頼

設計原則:

- ・ **CTRL は冪等**: 取りこぼしてもすぐ次が来るため, パケロス耐性は通信方針側で確保される.
- ・ **BEAT は即時**: 1 拍 1 発が原則. 確実性を上げる場合は同一 beatNo を 2~3 連送し, 受信側で beatNo 重複排除する (冪等処理).
- ・ **楽譜側内挿**: Version 0.2 で SelfRun 自走は撤廃. BEAT が長く来なくても楽器は Playing に留まり, 次の BEAT 到来時に自然再開する. 連送 (beatRedundancy) と過去 BEAT の即発火で取りこぼしを吸収する.
- ・ **NOTE は単一経路**: 楽器 → 自パート Mac で 1 系統. 楽譜に従って必ず送出する. Arduino-Mac 間は 1 対 1 USB Serial 直結のため, パケロスやネットワーク輻輳の心配なく確定到達するのが利点.

1.3.4 通信方式 (WiFi UDP + USB Serial の二系統)

通信仕様を表 1.5 に示す. 運用時に変わる値 (SSID / パス) は OrcNetConfig のリテラルで一元管理する.

表 1.5: 通信仕様

項目	設計
ノード間 (CTRL/BEAT)	WiFi UDP. node_01 → node_02-05 へ UDP/5001 マルチキャスト (仮 239.0.0.1). 楽 器 側 は <code>WiFiUDP.beginMulticast()</code> でグループに join

項目	設計
WiFi モード	node_01 が SoftAP (<code>WiFi.beginAP()</code>), node_02-05 が STA (<code>WiFi.begin()</code>) として接続. 外部ルータ・テザリング 不要
SSID / パス	仮 <code>OrchestraAP / orchestra2026</code> (node_01 が公開し , node_02-05 が同設定で接続). 本番運用で差替え
IP 割り当て	node_01 は SoftAP の既定 <code>192.168.4.1</code> . node_02-05 は SoftAP 内蔵 DHCP で <code>192.168.4.2--5</code> を自動取得
Arduino-Mac (NOTE)	間 USB Serial (CDC, <code>Serial.write</code>) . 各楽器ノードと専用 Mac が 1 対 1 で USB 接続. Mac 側 Processing は対応する 1 ポートだけを開く
ボーレート	<code>115200 bps</code> (USB CDC は実質高速だが値は揃える). <code>NoteSenderConfig.baudRate</code> で集約
フレーミング	全パケットの先頭 12 B 共通ヘッダの <code>magic = 0x4F52</code> がフレーム同期マークを兼ねる (§2.3.3)
エンディアン	リトルエンディアン (ESP32-S3 Xtensa LX7 / RA4M1 Cortex-M4 と同一)

1.3.5 パケロス・ジッタ対策

- ・ **CTRL 取りこぼし**: 冪等かつ 20 Hz 定期送信なので設計上問題にならない.
- ・ **BEAT 取りこぼし**: 同一 BEAT を 2~3 連発 + 楽器側で `beatNo` 重複排除. `SelfRun` (自走による仮想 BEAT 内挿) はテスト実装で「鳴らない症状」の温床となったため Version 0.2 で撤廃した (§2.5.4). BEAT が長く来なくても楽器は `Playing` に留まり, 次の BEAT 到来時に自然再開する.
- ・ **ジッタ (楽器間同期誤差)**: BEAT 受信時刻ではなく**マスタ時刻** `playAtMasterMs` で**発音タイミングを揃える** (§2.3.3.6). WiFi の到達時刻ばらつきは時計オフセット推定の誤差に押し付けられ, 4 台の発音タイミングは収束する.
- ・ **プロトコル不整合**: `magic / version` 不一致のパケットは破棄し, `StatusLedModule` が `data.orcNet` を読んで点滅パターンに反映する.

1.4 状態遷移図

各ノード種ごとの演奏状態を 4 状態の有限状態機械として定義する。状態は ConductorState（指揮者）と PerformerState（楽器）の enum class として SystemData の中に保持し、applyPattern() が遷移を判定する。

1.4.1 指揮者ノード (node_01)

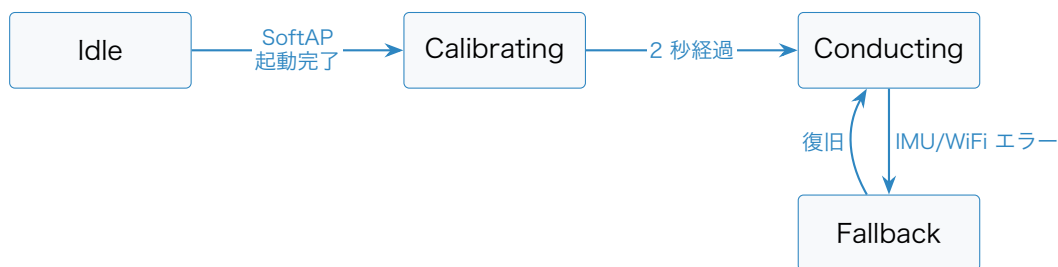


図 1.2 指揮者ノードの状態遷移

表 1.6: 指揮者ノードの状態定義

状態	意味	動作
Idle	起動直後、SoftAP 起動前	ImuModule.updateInput は動くが拍検出は無効、OrcSenderModule は何も送らない
Calibrating	初期静止姿勢の学習中（2 秒）	applyPattern() が静止時の加速度ノルムを平均し gravityMag (≈1 g) を記録
Conducting	通常演奏中	全モジュール稼働、BEAT を拍検出時に送出、CTRL を 20 Hz 周期で送出
Fallback	エラー発生	CTRL は state=Fallback で送り続ける、BEAT は停止、LED 高速点滅

1.4.2 楽器ノード (node_02～05)

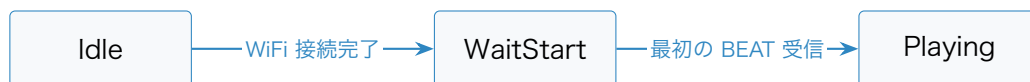


図 1.3 楽器ノードの状態遷移 (Version 0.2 で SelfRun を撤廃した 3 状態構成)

表 1.7: 楽器ノードの状態定義

状態	意味	動作
Idle	起動直後, node_01 SoftAP 未接続	受信モジュールは動くが楽譜進行ロジックは無効, SoftAP に接続できるまでリトライ
WaitStart	SoftAP 接続済, 最初の BEAT 待ち	CTRL/BEAT を受信し時計オフセットの EMA 学習を開始する. 収束待ちで Playing 遷移をブロックすると「鳴らない」症状の温床となるため, 時計同期収束 / startBeatNo の判定はここでは行わない (最初の数拍は受信遅延ぶんのズレが残るが, CTRL を数発受けたあとに自然収束する)
Playing	通常演奏中	BEAT 受信ごとに楽譜 currentEventIndex を 1 個進めて NOTE を送出. BEAT が長く来なくても Playing に留まり, 次の BEAT で自然再開する

第 2 章 システムの詳細設計

本章では、第 1 章で示した基本設計を実装可能な水準まで具体化する。塩澤担当範囲 (WBS タスク 121 「共通層 API 詳細」、122 「指揮者ノード詳細」、123 「楽器ノード詳細」) の 3 領域を中心に、部品・ハードウェア配線・ソフトウェア構造・処理フロー・テストを順に記す。コード例は要点抽出にとどめ、フル実装は本書スコープ外とする。

2.1 部品一覧

製品分解構造 (PBS) として、本ファームウェアが直接関わるハードウェア・ソフトウェア構成要素を表 2.1・表 2.2 に列挙する。PC 側 Processing 環境やスピーカは本書スコープ外であるが、NOTE パケットの受信境界として記述する。

表 2.1: ハードウェア部品一覧 (PBS)

区分	要素	数量	備考
P1.1 指揮者	XIAO ESP32-S3 Sense	1	Espressif ESP32-S3R8 (Xtensa LX7 デュアルコア, 8 MB PSRAM/Flash). OV2640 / PDM マイクは本案件未使用 (将来拡張余地)
P1.1	MPU6050 (GY-521 モジュール)	1	6 軸 IMU (加速度 $\pm 2/4/8/16$ g + 角速度 $\pm 250/500/1000/2000$ dps). 本案件では ± 4 g / ± 2000 dps. I2C アドレス 0x68. GY-521 はピンヘッダ・LDO・プルアップを実装したブレイクアウト基板で、3.3 V / GND / SDA / SCL / ADO のみ配線すれば動作する
P1.1	内蔵 WiFi (ESP32-S3 直制御)	1	WiFi.h (Arduino-ESP32) で SoftAP + UDP. AT bridge を経由しないため信頼性が高い
P1.1	状態 LED	1	LED_BUILTIN (XIAO 基板のユーザ LED) 流用

区分	要素	数量	備考
P1.2 楽器	Arduino UNO R4 WiFi	4	node_02-05 同一 H/W. 本番想定は 金管 1 / 金管 2 / 金管 3 / ドラム . Version 0.2 時点のテスト実装は node_02 = 金管 1, node_03 = 金管 2, node_04 = 木管 1 (音域分離の試行) として 3 台で動作確認済, node_05 (ドラム) は未実装
P1.2	内蔵 WiFi	4	WiFiS3, node_01 SoftAP に STA として接続
P1.2	状態 LED	4	LED_BUILTIN 流用
P2 ネットワーク	(node_01 SoftAP で兼用)	0	外部 AP・ルータ・テザリングは使用しない. node_01 が WiFi.softAP() で AP を起動し, 楽器が STA で接続する
P3 給電・通信	USB Type-C ケーブル	5	楽器 4 本は 各 Mac へ 1 対 1 直結 (給電 + Serial 通信兼用). 指揮者ノードは USB ハブ経由で給電のみ
P4 境界 (参考)	Mac (Processing)	4	本書スコープ外. 各楽器から USB Serial で NOTE を受信し, 倍音・ADSR 調整・波形生成を行う
P4 境界 (参考)	内蔵スピーカ	4	本書スコープ外. Mac 1 台あたり 1 系統

表 2.2: ソフトウェア技術スタック

レイヤ	採用技術	バージョン / 備考
MCU (指揮者)	XIAO ESP32-S3 Sense	ESP32-S3R8 (Xtensa LX7 240 MHz × 2 / 8 MB PSRAM / 8 MB Flash)
MCU (楽器)	Arduino UNO R4 WiFi	Renesas RA4M1 (Cortex-M4 48 MHz) + ESP32-S3 (WiFi 担当)
フレームワーク 言語	Arduino Framework C++ (C++11)	PlatformIO 経由 arduino-core 範囲
ビルド (指揮者)	PlatformIO	platform=espressif32, board=seeed_xiao_esp32s3

レイヤ	採用技術	バージョン / 備考
ビルド（楽器）	PlatformIO	platform=renesas-ra, board=uno_r4_wifi
外部ライブラリ（指揮者）	Wire（標準同梱）	I2C ドライバを直接叩いて MPU6050 の PWR_MGMT_1/ACCEL_CONFIG/GYRO_CONFIG/ACCEL_XOUT_H を読み書き. Adafruit_MPU6050 等の高水準ライブラリは取り回しの軽さを優先して使わない (§2.5.4)
WiFi スタック（指揮者）	WiFi.h（Arduino-ESP32 / ESP-IDF）	SoftAP + UDP マルチキャスト/ブロードキャストをネイティブサポート
WiFi スタック（楽器）	WiFiS3	標準同梱. STA 接続 + UDP 受信
設計パターン	EMA	全章前提（ADR-0005）
共通層（流用）	ModuleCore（IModule + ModuleTimer）	EMA からそのまま流用. HW 差は ImuModule と OrcNetModule に局在
自作層（新規）	OrcProtocol OrcNetModule	/ §2.3.3 で定義. ESP32-S3 / RA4M1 でビルド分岐
ネットワーク	UDP over WiFi（ノード間）+ USB Serial（PC 間）	二系統に分離

2.2 ハードウェア設計

2.2.1 配線・ピン

指揮者ノード（node_01: XIAO ESP32-S3 Sense）は外付け MPU6050 を I2C で接続する. XIAO の I2C 既定ピンは SDA=D4（GPIO5）, SCL=D5（GPIO6）. Sense 拡張機能の PDM マイクは D11/D12 を専有するが本案件では未使用. 3.3 V / GND を MPU6050 に給電し, AD0 を GND に落として I2C アドレスを 0x68 に固定する. 楽器ノード（node_02-05: UNO R4 WiFi）は内蔵 LED 以外に外部配線を要しない（表 2.3）.

表 2.3: 配線・ピンアサイン

ノード	用途	ピン
node_01	I2C SDA (MPU6050)	D4 (GPIO5, I2C_SDA_PIN = 5)
node_01	I2C SCL (MPU6050)	D5 (GPIO6, I2C_SCL_PIN = 6)
node_01	MPU6050 電源	3V3 / GND. AD0 を GND に落とし I2C アドレスを 0x68 に固定
node_01	状態 LED	LED_BUILTIN (XIAO ユーザ LED)
node_01	給電	USB Type-C (USB ハブ経由, Serial は使用しない)
node_02-05	状態 LED	LED_BUILTIN
node_02-05	給電 + NOTE 送信	USB Type-C (各楽器が専用 Mac へ 1 対 1 直結. Serial.write で NOTE 出力)

2.2.2 ネットワーク構成

通信は二系統に分離する。Arduino 同士は WiFi UDP (node_01 自身が立てる SoftAP 配下) で, Arduino と Mac は USB Serial 直結で通信する。node_01 (XIAO ESP32-S3 Sense) は ESP-IDF を介した `WiFi.softAP("OrchestraAP", "orchestra2026")` で SoftAP を起動し, node_02-05 (UNO R4 WiFi) は `WiFiS3` で同 SSID/パスへ STA として接続する。SoftAP の既定アドレスは 192.168.4.1, 楽器側は内蔵 DHCP で 192.168.4.2--5 を取得する。node_01 は CTRL/BEAT を **UDP マルチキャスト** (仮 239.0.0.1:5001) で送信し, node_02-05 はそれぞれグループに join して受信する。NOTE は各楽器ノードが **専用 Mac のシリアルポート** へ `Serial.write` で書き込む (楽器 ↔ Mac は 1 対 1)。本番運用時の SSID / パスは `OrcNetConfig` のリテラルで切替可能とする。Mac 側の Processing は自分に繋がっているシリアル 1 ポートだけを開いて受信する。

採用判断: 指揮者だけを ESP32-S3 直制御の XIAO に置き換えたのは, UNO R4 WiFi の `WiFiS3` が AT bridge を経由する都合で SoftAP + マルチキャストの実機動作にコミュニティ報告レベルの不安があったためである。ESP32-S3 を Arduino-ESP32 で直接制御すれば `WiFi.softAP()` と `WiFiUDP` がネイティブにサポートされ, ESP-NOW などの代替手段 (ストレッチ拡張) も選択できる。**利点:** 外部ルータ・テザリングが不要なため, 会場 LAN のセキュリティ制限や電波混雑の影響を受けない。

STA 側の注意: ESP32-S3 SoftAP は仕様上, 接続中 STA が省電力モードのときマルチキャストパケットをキャッシュしない。したがって楽器側 (UNO R4 WiFi) の WiFi 省電力は無効化 (`WiFi.lowPowerMode()` を呼ばない) して運用する。

2.2.3 物理配置

演奏時は指揮者を中心に楽器 4 台を扇形に配置し、各 Mac とそれぞれの内蔵スピーカは観客側からは見えない位置に並べる（楽器 → Mac は USB ケーブルで直結）。指揮者ノードの給電は USB ハブで集約する。配置に関する具体寸法・治具設計はチーム全体側（梅澤担当）に委ねる。

2.3 ソフトウェア設計

2.3.1 ファイル構成

firmware/ 配下は **共通層**と**ノード別プロジェクト**の 2 段構造にする。EMA に準拠し、PlatformIO の `lib_extra_dirs` で共通層を全ノードから参照する。

firmware/ ディレクトリ構成（抜粋）

```
firmware/
|-- common/
|   |-- lib/                                # 全ノード共有 (lib_extra_dirs で参照)
|       |-- ModuleCore/                    # コア層 (プロジェクト非依存)
|           |-- IModule.h
|           |-- ModuleTimer.h
|           |-- OrcProtocol/                # CTRL/BEAT/NOTE のパケット定義
|               |-- OrcProtocol.h
|               |-- OrcProtocol.cpp
|           |-- OrcNetModule/              # WiFi 接続維持 + UDP 送受信
|               |-- OrcNetModule.h
|               |-- OrcNetModule.cpp
|           |-- StatusLedModule/           # 出力: 状態 LED (両ノード種で共有)
|               |-- StatusLedModule.h
|               |-- StatusLedModule.cpp
|           |-- SerialDebug/               # デバッグ出力ヘルパ (SERIAL_DEBUG で切替)
|               |-- SerialDebug.h
|-- node_01/                               # 指揮者ノード
|   |-- platformio.ini
|   |-- include/
|       |-- SystemData.h                  # {Module}Data を集約
|       |-- ProjectConfig.h               # {MODULE}_CONFIG + 共有バスピン
|   |-- src/
|       |-- main.cpp                      # 入出力配列・3 フェーズループ
|       |-- applyPattern.cpp              # フィルタ/拍検出/テンポ推定/状態遷移
|   |-- lib/
|       |-- ImuModule/                    # 入力: IMU
|       |-- OrcSenderModule/              # 出力: CTRL/BEAT 送信
|-- node_02/                               # 楽器ノード A
|   |-- platformio.ini
|   |-- include/
```

```

| | |-- SystemData.h
| | |-- ProjectConfig.h          # partId=0x02, PC IP, startBeatNo 等
| | `-- score_data.h           # パート A の楽譜 (ScoreEvent 宣言)
| |-- src/
| | |-- main.cpp
| | |-- applyPattern.cpp        # 楽譜進行 (index 駆動 + 細分音符予約)
| | `-- score_data.cpp         # パート A の楽譜本体 (c 配列)
| `-- lib/
|     |-- OrcReceiverModule/    # 入力: CTRL/BEAT 受信
|     `-- NoteSenderModule/    # 出力: NOTE 送信 (USB Serial)
|-- node_03/ (構造同じ。ProjectConfig と score_data.* が差分)
|-- node_04/
`-- node_05/

```

各ノードの platformio.ini には次の 2 行を必ず含める。-I include は EMA の必須設定 (lib/ から include/SystemData.h を参照するため), lib_extra_dirs は本プロジェクト固有の設定 (5 ノードで firmware/common/lib/ を共有するため)。

```

build_flags    = -I include          ; lib/ から include/SystemData.h を参照するために必須
lib_extra_dirs = ../common/lib      ; ModuleCore / OrcProtocol / OrcNetModule を共有

```

2.3.2 オブジェクト設計 (クラス図)

EMA の IModule 抽象基底をすべてのハードウェアモジュールが継承する。OrcNetModule と StatusLedModule は共通層 (firmware/common/lib/) に置き両ノード種で共有する (図 2.1)。

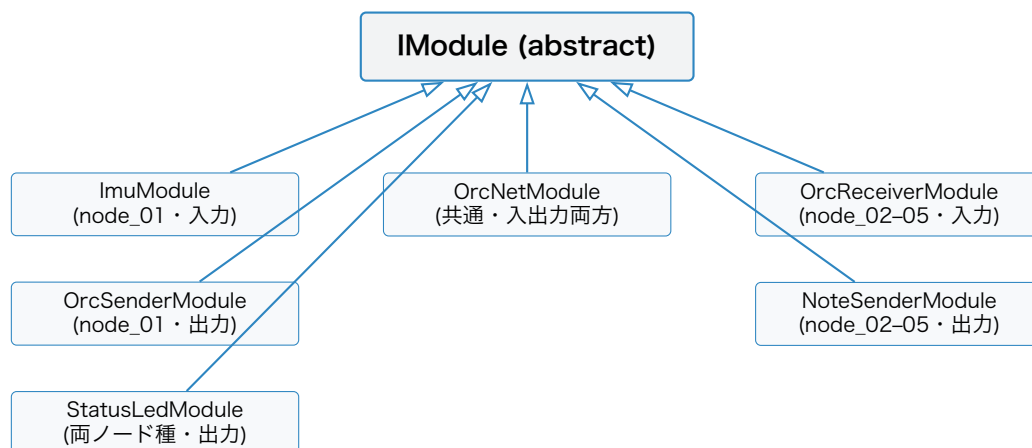


図 2.1 モジュールのクラス継承関係

表 2.4: モジュール一覧（入出力分類別）

モジュール	ノード	分類	責務
ImuModule	node_01	入力	MPU6050 (I2C) の 6 軸を読み data.imu に書く
OrcSenderModule	node_01	出力	data.beat/data.tempo を読み CTRL/BEAT 送信キューに投入
OrcReceiverModule	node_02-05	入力	orcNet.lastCtrl/lastBeat を読み, 時計オフセット EMA・受理 BEAT キュー・CTRL 状態を data.sync / data.receiver / data.ctrl に整形
NoteSenderModule	node_02-05	出力	noteOut.pendingOn を見て NoteOn を Serial.write で PC へ送信. NoteOff は送らず Processing 側が durationMs で自動消音
OrcNetModule	node_01・ node_02-05	入出力	WiFi 接続維持, UDP 受信ポーリング (入力) と CTRL/BEAT 送信キューフ ラッシュ (出力)
StatusLedModule	全ノード	出力	状態に応じた LED 点滅パターン表示

2.3.3 関数設計

2.3.3.1 IModule インターフェース (流用)

EMA のすべての入出力モジュールが継承する基底クラス. 3 フェーズループから一律に呼ばれる入口を定める. 依存を避けるため SystemData は前方宣言で参照する.

IModule (モジュール抽象基底)

メンバ

シグネチャ	役割
enabled	有効/無効フラグ. init() 失敗時に false に落とし、ループ側はこれを見て呼び出しをスキップ
init()	ハードウェア初期化. 成功で true, 失敗で false を返す
updateInput(data)	入力フェーズで呼ばれる. センサ値や受信結果を data に書く
updateOutput(data)	出力フェーズで呼ばれる. data の値をハードウェア/送信に反映
deinit()	後始末. 基底側は空実装

init() の失敗時は setup() 側で MAX_RETRY 回リトライし、それでも失敗した場合は enabled = false とする. ロジックフェーズで定期的に再 init() を試みて enabled = true に復帰させる (§2.4.2 参照).

2.3.3.2 ModuleTimer (流用)

「ある基準時刻からの経過 ms」を返す軽量タイマ. 内部に基準時刻を 1 つ持つだけの薄いラップで、周期判定やタイムアウト判定に使う.

ModuleTimer (経過時間タイマ)

メンバ

シグネチャ	役割
setTime(offsetMs = 0)	基準時刻を「現在 - offsetMs」にセット. 引数省略で「いま」を起点にする
getNowTime()	基準時刻からの経過 ms を返す

用途例: IMU サンプリング 200Hz (intervalMs = 5), CTRL 送信 20Hz (50ms), LED 点滅周期, BEAT 不応期, CTRL 送信周期.

2.3.3.3 通信パケット定義 (OrcProtocol)

3 種のパケットすべてに 12 バイトの共通ヘッダを付与する. データはリトルエンディアンでシリアライズする. CTRL/BEAT は WiFi UDP マルチキャスト, NOTE は USB Serial (バイナリ列を直接書込) で転送する. magic = 0x4F52 は WiFi UDP ではプロトコル識別子, USB Serial では **フレーム同期マーク**を兼ねる (Mac 側は magic を探して同期し, type でペイロード長を判定して残りを読む).

共通ヘッダ

全 12 バイト. 全パケットの先頭に付与する.

オフセット	フィールド	型	説明
0	magic	uint16_t	固定値 0x4F52 (ASCII "OR")
2	version	uint8_t	初期値 0x01
3	type	uint8_t	1=CTRL, 2=BEAT, 3=NOTE
4	seq	uint32_t	単調増加 (パケロス検出用)
8	timestampMs	uint32_t	マスタ時刻 (= 指揮者ノードの millis()). 楽器側は受信時刻と比較してオフセット推定に使う (§2.3.3.6)

CTRL ペイロード

type = 1. 定期 20 Hz 送信. 共通ヘッダ含め全 20 バイト.

オフセット	フィールド	型	説明
12	bpmQ8	uint16_t	BPM × 8 (例: 120.5 → 964)
14	velocity	uint8_t	0-127. ストレッチ未実装時は固定 64
15	state	uint8_t	0=Idle, 1=Calibrating, 2=Conducting, 3=Fallback
16	予約	uint8_t[4]	ゼロ埋め

BEAT ペイロード

type = 2. 拍検出時イベント送信. 共通ヘッダ含め全 20 バイト.

オフセット	フィールド	型	説明
12	beatNo	uint16_t	曲頭からの拍番号 (曲開始時に 0 リセット)
14	予約	uint8_t[2]	ゼロ埋め
16	playAtMasterMs	uint32_t	マスタ時刻でこの ms に発音せよ という未来時刻指定. 送信時に masterNow + lookahead で算出

NOTE ペイロード

type = 3. 楽譜イベント発火時に USB Serial へ送信. 共通ヘッダ含め全 20 バイト.

オフセット	フィールド	型	説明
12	partId	uint8_t	0x02-0x05 (= node_02-05)
13	noteNumber	uint8_t	MIDI ノート番号 (0-127, 60=C4)
14	velocity	uint8_t	0-127
15	gate	uint8_t	1=NoteOn, 0=NoteOff
16	durationMs	uint16_t	発音予定長さ (参考値)
18	予約	uint8_t[2]	ゼロ埋め

2.3.3.4 OrcNetModule API (WiFi UDP 専用)

WiFi 接続維持と UDP 送受信ポーリングを集約する IModule 実装. CTRL/BEAT のみを担当し, NOTE は別モジュール (NoteSenderModule) が USB Serial で直送する. 入出力両方の責務を持つため, inputModules[] と outputModules[] の両方に登録する.

WifiMode (列挙)

値	意味
SoftAp	自身がアクセスポイントを起動する側 (指揮者ノード node_01)
Sta	既存 SoftAP に接続する側 (楽器ノード node_02-05)

OrcNetConfig (生成時に渡す設定)

フィールド	役割
mode	動作モード. SoftAp (指揮者) / Sta (楽器)
ssid	SoftAp なら自身が公開する SSID. Sta なら接続先 SSID
pass	WPA2-PSK パスフレーズ
multicastIp	CTRL/BEAT 配信先マルチキャスト IP (例: 239.0.0.1)
udpPort	CTRL/BEAT 共通ポート (既定 5001)
channel	SoftAp 時のみ参照 (既定 6)
reconnectIntervalMs	切断検出後の再接続試行間隔 (既定 2000 ms)

OrcNetData (SystemData.orcNet に置く受け渡し領域)

受信側 (他モジュールが読む)

フィールド	役割
wifiConnected	WiFi 接続が確立しているか
hasNewCtrl	今フレームで新しい CTRL を受信したか (読み手が消費後にクリア)
hasNewBeat	今フレームで新しい BEAT を受信したか (同上)
lastCtrl	直近に受信した CTRL ペイロード
lastBeat	直近に受信した BEAT ペイロード

送信側 (送信したいモジュールが書く)

フィールド	役割
hasPendingCtrl	未送信 CTRL がある (送出後に OrcNetModule がクリア)
pendingCtrl	送信したい CTRL ペイロード
hasPendingBeat	未送信 BEAT がある (同上)
pendingBeat	送信したい BEAT ペイロード
pendingBeatRedundancy	同一 BEAT を何発まで連送するか (既定 1)

OrcNetModule (クラス)

シグネチャ	役割
OrcNetModule(config)	コンストラクタ. OrcNetConfig を保持
init()	SoftAp なら WiFi.beginAP(), Sta なら WiFi.begin() の後に Udp.beginMulticast() を呼ぶ
updateInput(data)	WiFi 状態を data.orcNet.wifiConnected に反映. UDP 受信をポーリングして lastCtrl/lastBeat と hasNewCtrl/hasNewBeat に書く
updateOutput(data)	hasPendingCtrl/Beat が立っていれば該当パケットをマルチキャスト送出. pendingBeatRedundancy 回連送した後にフラグをクリア
deinit()	WiFi / UDP 後始末

ルール (EMA「モジュール間直接呼び禁止」に準拠) : 送受信ともに OrcNetModule の公開メソッドを他モジュールから直接呼ばない. 送信側は OrcSenderModule (楽器

側は `applyPattern()` が `data.orcNet.pendingCtrl/pendingBeat` と `hasPendingCtrl/Beat` を立てるだけで、実際の `Udp.send()` は `OrcNetModule.updateOutput()` が一括処理する。受信側も `tryRecvXxx()` のような外向き API を持たず、`OrcNetModule.updateInput()` が `lastCtrl/lastBeat` と `hasNewCtrl/hasNewBeat` に書き込み、他モジュールはこれを読むだけ。モジュール間の結合は `SystemData` の一点に閉じる。

2.3.3.5 NoteSenderModule API (USB Serial 専用)

楽器ノード (node_02-05) が NOTE をハードウェア Serial へ書き出すための IModule 実装。 `init()` で `Serial.begin(baudRate)` を呼び、`updateOutput()` で `data.noteOut.pendingOn` フラグを見て `NotePacket (gate = 1)` を構築 → 20 バイトのバイナリを `Serial.write()` で一括送出する。Version 0.2 では `NoteOff` パケットは送らない (§2.5.4) ため、`NoteOutData` は `pendingOff` を持たず `pendingOn` のみ。消音は Mac 側が `durationMs` を見て行う。

NoteSenderConfig (生成時に渡す設定)

フィールド 役割

<code>baudRate</code>	USB CDC ボーレート (既定 115200)
<code>partId</code>	自パート識別子 (0x02-0x05)。1 対 1 USB 接続のため Mac 側では参考値

NoteOutData (SystemData.noteOut に置く発音指示。applyPattern() が書き NoteSenderModule が読む)

フィールド 役割

<code>pendingOn</code>	未送信の <code>NoteOn</code> がある (<code>NoteSenderModule</code> が送出後にクリア)。Version 0.2 では <code>pendingOff</code> を持たない (<code>NoteOff</code> を送らないため)
<code>noteNumber</code>	MIDI ノート番号
<code>velocity</code>	0-127 (楽譜 <code>velocity</code> と CTRL <code>velocity</code> の乗算合成済み)
<code>durationMs</code>	発音予定長。Processing 側はこの値で自動消音する

NoteSenderData (送信制御の内部状態)

フィールド 役割

<code>noteSeq</code>	送信した NOTE の連番。共通ヘッダの <code>seq</code> に積む
<code>lastSentMs</code>	直近の <code>Serial.write()</code> 完了時刻

NoteSenderModule (クラス)

シグネチャ	役割
NoteSenderModule(config)	コンストラクタ
init()	Serial.begin(baudRate) を実行
updateOutput(data)	data.noteOut.pendingOn を見て NotePacket (gate = 1) を組み立て、20 バイトを Serial.write() → Serial.flush() で送出。送出後に pendingOn をクリア。NoteOff は送らない

Mac 側 Processing は、自パートの楽器ノードに繋がっている 1 つの USB シリアルポートを開き、magic 0x4F52 を探してフレーム同期し、type=3 のとき NOTE ペイロード 8 バイトを読み取って音色合成（倍音・ADSR 調整 → 波形生成）に渡す。partId は 1 対 1 接続のため検証用途で参照する。NoteOn 1 個に durationMs が乗っているので、Mac 側は durationMs 経過時点で当該ボイスを自動 release する（NoteOff パケットは受け取らない）。Version 0.2 のテスト用 Processing (pc_app/test/orchestra_player) はこの方式をサイン波 + 線形 ADSR・最大同時発音 16 (§2.5.4) で実装している。

2.3.3.6 マスタクロック方式と時刻同期

WiFi UDP マルチキャストは PHY のベーシックレートで配信されるうえ、UNO R4 WiFi は RA4M1 と内蔵 ESP32-S3 が SPI ブリッジで連結された 2 チップ構成のため、4 台の楽器が BEAT を受信してアプリ層に到達する時刻には数 ms～十数 ms のジッタが乗る。これをそのまま発音に使うと楽器間同期誤差（MOP-1）が運次第になる。本設計ではこのジッタを**時計同期の誤差に押し付ける**ことで、楽器間の相対同期をほぼゼロに収束させる。

基本アイデア

指揮者ノード (= SoftAP) の millis() を **マスタ時刻** として全ノードで共有する。BEAT は「**マスタ時刻 playAtMasterMs に発音せよ**」という **未来時刻指定**の指示として送る。楽器は自分の millis() をマスタ時刻に変換するためのオフセットを学習し続け、masterNow がその時刻に到達した瞬間に発音する。WiFi の到達時刻がばらついても、4 台が見ている共通のマスタ時刻に揃うため発音タイミングは合致する。

オフセット推定（楽器側）

共通ヘッダの timestampMs を「送信時のマスタ時刻」と読み替える。楽器は CTRL/BEAT を受信するたびに、自ノード時計とのズレを 1 サンプルとして取り出し、係数 $\alpha = 0.10$ の指数移動平均（EMA）で offset を更新する。

オフセット推定アルゴリズム

ステップ 処理内容

受信時	<code>sample = pkt.header.timestampMs - millis()</code> (マスタ時刻と自時計の差)
更新	<code>offset ← offset + α · (sample - offset)</code> ($\alpha = 0.10$)
発音判定	<code>masterNow = millis() + offset.</code> 保 留 BEAT の <code>playAtMasterMs</code> を超えたら発火

CTRL は 20 Hz で常時流れるためサンプル数は十分. EMA 5-10 サンプルで収束する. 片方向放送のため WiFi の片道遅延がオフセットに系統誤差として乗るが, 4 台でほぼ同じ値だけずれるので楽器間相対同期には影響しない (MOP-1 の評価軸はこの相対同期).

lookahead と期限切れ挙動 (Version 0.2)

指揮者は BEAT 送出時に `playAtMasterMs = masterNow + beatLookaheadMs` を計算する (`beatLookaheadMs` は `ORC_SENDER_CONFIG` で集約, 仮値 50 ms). 楽器側は次の挙動で安全側に倒す:

- ・ **未来時刻** (`playAtMasterMs > masterNow`): 通常, `masterNow` 到達まで待機.
- ・ **過去時刻: 即発火 (捨てない)**. Version 0.1 では `expiredGraceMs` (100 ms) を超えたらスキップする設計だったが, テスト実装で「指揮者の WiFi リトライや楽器側の起動直後で BEAT を捨てると鳴らない」症状が頻発したため, 0.2 では遅延量に関わらず即発火に倒した. 代わりに `beatRedundancy = 2` で同一 BEAT を冗長送信し, どれか 1 発が `lookahead` 内に届けば「ほぼ揃う」状態を作る.

将来 MOP-1 評価で「過去 BEAT の連発が誤差として無視できない」ことが分かれば, `expiredGraceMs` の概念を再導入する余地はある. 現時点では「鳴らない」リスクの方が上位の障害として優先する.

同期確立フェーズ (Version 0.2 で簡素化)

楽器ノードの `WaitStart` → `Playing` 遷移条件は「**最初の BEAT を受信したら**」とする. 時計同期が収束してから `Playing` に入る設計 (0.1) は, CTRL 数発の収束待ちで「鳴らない」温床となるため取り下げた. 代わりに 0.2 では「最初の数拍は受信遅延ぶんのズレが残るが, CTRL を数発受けたあと `offset` が EMA で自然収束する」運用とする. MOP-1 の評価は同期収束以降の定常状態で行う. `clockSyncMinSamples` はデバッグ表示用フラグに格下げ.

設計上の固定遅延 (受け入れ判断)

`beatLookaheadMs = 50 ms` は「指揮の振り下ろしから音が出るまでの固定遅延」としてそのまま乗る. 人間の視聴覚同時知覚閾は約 80 ms (聴覚が遅れる方向) なので 50 ms は知覚

されない範囲だが、ハッカソン用途では受け入れる設計判断とする。振り下ろし最下点予測などの高度化は本書スコープ外。

2.3.3.7 命名規則・ログ書式 (EMA 準拠)

- ・ Config 型: {Module}Config (例: ImuConfig)
- ・ Config インスタンス: {MODULE}_CONFIG (例: IMU_CONFIG)
- ・ Data 型: {Module}Data (例: ImuData)
- ・ ログ: [ModuleName] msg 形式 (例: [OrcNet] reconnecting (attempt 3))
- ・ デバッグログは #ifdef DEBUG で切替え可能

2.4 処理フローの設計

2.4.1 共通: 3 フェーズ実行モデル

全ノード共通で、loop() は「入力 → ロジック applyPattern() → 出力」の 3 フェーズを毎周期この順で実行する。enabled == false のモジュールはスキップする。

loop() の実行順序

フェーズ	処理内容
1. 入力	inputModules[] を先頭から順に updateInput(systemData) で呼び出す。センサ値や UDP 受信結果が SystemData に書き込まれる
2. ロジック	applyPattern(systemData) を 1 度だけ呼び、フィルタ／拍検出／状態遷移／楽譜進行などはすべてここに集約
3. 出力	outputModules[] を先頭から順に updateOutput(systemData) で呼び出す。LED 表示・UDP 送信・Serial 送出自が反映される

入力モジュールは SystemData を書き込み、出力モジュールは読み込む。クロス参照は禁止。OrcNetModule は両配列に登録し、「入力フェーズで先に受信、出力フェーズの最後で送信」を配列順で制御する。

2.4.2 指揮者ノード (node_01)

2.4.2.1 モジュール構成

- ・ 入力配列: OrcNetModule, ImuModule
- ・ 出力配列: OrcSenderModule, StatusLedModule, OrcNetModule
- ・ ロジック: applyPattern() (フィルタ・拍検出・テンポ推定・状態遷移)

2.4.2.2 ProjectConfig (抜粋)

node_01/include/ProjectConfig.h に、共有バスピンと各モジュール設定インスタンス、ロジック層パラメータを集約する。具体値はこの 1 ファイルでだけ管理し、モジュール本体にはハードコードしない。

共有バスピン

定数	値	役割
I2C_SDA_PIN	5 (D4 = GPIO5)	XIAO ESP32-S3 Sense の I2C SDA 既定
I2C_SCL_PIN	6 (D5 = GPIO6)	同上 SCL

IMU_CONFIG

キー	値	役割
address	0x68	MPU6050 I2C アドレス (ADO=GND). ADO=VCC なら 0x69
sampleIntervalMs	5	200 Hz サンプリング
accelRangeG	4	加速度フルスケール (2/4/8/16 g 選択可)
gyroRangeDps	2000	ジャイロフルスケール (250/500/1000/2000 dps 選択可)

ORC_NET_CONFIG (指揮者)

キー	値	役割
mode	SoftAp	node_01 自身が AP を起動
ssid	"OrchestraAP"	公開 SSID
pass	"orchestra2026"	WPA2-PSK
multicastIp	239.0.0.1	CTRL/BEAT 配信先
udpPort	5001	共通ポート
channel	6	SoftAp チャンネル

ORC_SENDER_CONFIG

キー	値	役割
ctrlIntervalMs	50	CTRL 送信周期 (20 Hz)
beatRedundancy	2	同一 BEAT の連送回数 (lookahead 期限切れ回避)
beatLookaheadMs	50	マスタ時刻で何 ms 先を発音指定にするか

STATUS_LED_CONFIG

キー	値	役割
pin	LED_BUILTIN	表示ピン
blinkIntervalMs	500	既定点滅周期

LOGIC_PARAMS (applyPattern()) のロジック係数

キー	値	役割
lpfAlpha	0.10	加速度 IIR LPF の係数
beatDynThresholdG	1.20	Armed 突入トリガ. 動加速度ノルム (LPF 後 - キャリブ重力) の閾値. 重力の下駄が引かれているので「純粋な振り下ろし加速度」
beatRefractoryMs	350	拍検出後の不応期 (約 170 BPM 上限)
beatFirePathM	0.20	早期発火閾値. Armed 中の 擬似経路長 (dynAcc を時間積分した vel の絶対値をさらに時間積分した値) がこの値に達した瞬間に拍を発火
beatArmedMinHoldMs	50	Armed 突入直後の単発ノイズで誤リリースしないための最低保持時間
beatArmedTimeoutMs	800	Armed が長引いた場合の強制終了. 普通の振り下ろしは 200-500 ms 程度
beatReleaseG	0.20	リリース判定 (絶対値). 動加速度ノルムがこの値未満なら停止と見なす
beatReleaseRatio	0.40	リリース判定 (相対値). Armed 中ピーク値の 40% を割ったら振り終わりと見なす. 連続スイングで dynNorm が 0 まで落ちきらないケースをこちらで救済
beatReleaseHoldMs	40	リリース判定のデバウンス. release 条件が連続成立した時間がこの値を超えてから真のリリースと認定 (5 ms 周期で 8 サンプル分)
bpmEmaAlpha	0.30	テンポ推定 EMA 係数
bpmMin	40.0	推定 BPM 下限クランプ
bpmMax	240.0	推定 BPM 上限クランプ
calibrationMs	2000	起動時キャリブレーション期間 (静止時の加速度ノルムを平均し, 静止ノルム gravityMag $\approx$ 1 g を確定する)
imuTimeoutMs	200	IMU が ready=false のまま継続したら Fall-back 遷移

Version 0.1 からの変更: 旧 `beatAccelThresholdG = 1.80` (重力込み) はキャリブ重力を引かない値で、重力 1g の下駄ぶん大きい振りでないとは届かなかった。0.2 では重力補正後の動加速度 (`dynNorm`) で判定するため、意味的に **1.80g - 1.0g ≈ 0.80g 相当** の判定に下げた上で、ピーク 1.5--3.0g の指揮ストロークから安定して Armed に入るバランスとして 1.20g に置いた。不応期は 250 ms から 350 ms に拡大し、1 振り中の「振り下ろし → 振り戻し」両端を別の拍として誤検出することを防ぐ。さらに「経路長閾値による早期発火」と「デバウンス付きリリース」を追加し、振り始めから約 100 ms で発火する応答性と、連続スイング時の二重発火耐性を両立させた。

2.4.2.3 SystemData (抜粋)

`ImuData`・`OrcNetData`・`OrcSenderData`・`StatusLedData` に加え、`applyPattern()` の中間状態として `BeatLogicData`・`TempoLogicData`・`CalibrationData`・`ConductorStateData` を集約する。

ConductorState (列挙)

値	意味
Idle	起動直後、未初期化
Calibrating	静止ノルム (≈ 重力 1g) の採取中 (起動から <code>calibrationMs</code>)
Conducting	通常動作、拍検出と CTRL/BEAT 送信を行う
Fallback	IMU / WiFi 異常時のフェイルセーフ

ロジック中間データ

構造体	主なフィールド／役割
BeatLogicData	event (今周期に拍を検出したか) / beatNo (曲頭からの拍番号) / lastBeatMs (直近検出時刻) / playAtMasterMs (送信予定マスタ時刻)
TempoLogicData	bpm (EMA 平滑後の推定 BPM) / nextBeatPredictedMs (次拍予測時刻) / velocity (CTRL に乗せる強度, 初期実装は固定 64)
CalibrationData	done (完了フラグ) / startMs (採取開始時刻) / sampleCount * accumNorm (採取中の累積) / gravityMag (確定した静止ノルム $\approx$ 重力 1g. 軸ごとのベクトルではなくスカラー 1 個)
ConductorStateData	state (現在の ConductorState)

SystemData (指揮者ノードの集約データ)

メンバ	役割
imu	IMU 読み取り値 (加速度・角速度・ノルム)
orcNet	OrcNetModule の受信結果と送信予約
sender	CTRL/BEAT のシーケンス番号など送信側の内部状態
led	StatusLedModule への指示値
beat	拍検出結果 (BeatLogicData)
tempo	テンポ推定結果 (TempoLogicData)
calibration	キャリブレーション進行状況 (CalibrationData)
conductor	状態遷移の現在状態 (ConductorStateData)

2.4.2.4 applyPattern() の処理フロー

毎周期, 以下の順で処理する.

1. **IIR LPF + ノルム計算 + 動加速度算出**: `data.imu.acc` を一次ローパス ($\alpha = 0.10$) で平滑化し `accLpf` を得て, そのノルム `accNorm` を計算する. **動加速度ノルム** `dynNorm` = $\max(0, \text{accNorm} - \text{gravityMag})$ を拍検出の判定対象とする. 加速度ベクトルではなく **ノルム** からスカラーの静止ノルム ($\approx 1\text{g}$) を引くため姿勢に依存せず, 水平で校正してから傾けて振っても残留重力で誤検出しない. Calibrating 完了前は `gravityMag` = 0

のため $\text{dynNorm} \approx \text{accNorm}$. 経路長の二重積分用に, accLpf の向きを保ったまま大きさを dynNorm にスケールした近似ベクトル $\text{dynAcc} = \text{accLpf} \times (\text{dynNorm} / \text{accNorm})$ も求める.

2. **拍検出 (ゲート式ステートマシン + 早期発火)** : `state == Conducting` のときのみ実行. 詳細は次項.
3. **テンポ推定 (EMA)** : 拍間隔 $\rightarrow$ 瞬時 BPM $\rightarrow \alpha = 0.30$ の指数移動平均で `data.tempo.bpm` を更新. `[bpmMin, bpmMax]` にクランプし `nextBeatPredictedMs` を計算.
4. **状態遷移**: `Idle` $\rightarrow$ `Calibrating` (SoftAP 起動完了) $\rightarrow$ `Conducting` (`calibrationMs` 経過) $\rightarrow$ `Fallback` (IMU/WiFi エラー) $\rightarrow$ `Conducting` (復旧) の判定. `Fallback` 判定は $(\text{now} - \text{imu.sampleAtMs}) > \text{imuTimeoutMs}$ または `!orcNet.wifiConnected` で発動.
5. **LED 反映**: `conductor.state` に応じて `led.solidOn` と `led.blinkIntervalMs` を設定.

拍検出アルゴリズム (Version 0.2)

0.1 では「ノルムピーク閾値 + 不応期」の単純判定だったが, テスト実装で次の不具合が判明した: (a) ピーク後に発火するため「振りと音」のラグが大きい, (b) 振り戻しを 2 拍目として拾う, (c) 連続スイングで dynNorm が完全に下がらず `Armed` が抜けない, (d) 軽い揺れでも誤発火する. これらを構造的に解消するため, 以下の **ゲート式ステートマシン** に作り直した.

拍検出ステートマシン (applyPattern() 内のローカル状態)

状態

`Idle` 動加速度が小さい状態. 積分しない (ノイズ蓄積を防ぐ)

`Armed` $\text{dynNorm} > \text{beatDynThresholdG}$ で突入. `Armed` 中だけ dynAcc を時間積分して擬似経路長 `sPathLen` を更新する

Idle $\rightarrow$ Armed の遷移

$\text{dynNorm} > \text{beatDynThresholdG}$ (1.20g) で `Armed` に入る. `sVel`, `sPathLen`, `sArmedPeakDyn` をリセットし, `sArmedAtMs = now` を記録.

Armed 中の発火 (早期発火)

1 `Armed` セッションに高々 1 回. 以下の AND で発火:

- ・ 未発火 (`!sBeatFiredInArmed`)
- ・ $\text{sPathLen} \geq \text{beatFirePathM}$ (0.20m)
- ・ $(\text{now} - \text{sLastBeatMs}) \geq \text{beatRefractoryMs}$ (350ms)

発火しても `Armed` は **維持** する. これが「1 振り = 1 拍」の鍵で, 「発火 $\rightarrow$ 即 `Idle` $\rightarrow$ 高 dyn のまま再 `Arm` $\rightarrow$ `refractory` 境界で再発火」という連続発火ループを構造的に防ぐ.

Armed → Idle の遷移（リリース判定）

以下のいずれかでリリース、beatArmedMinHoldMs（50ms）経過後に判定する。

- ・リリース条件 `releaseInst: dynNorm < beatReleaseG（0.20 g）` または `dynNorm < sArmedPeakDyn × beatReleaseRatio（0.40）`。単発のノイズで抜けないよう `beatReleaseHoldMs（40ms）` のデバウンスを掛け、`releaseInst` が連続成立した時間が閾値を超えて初めて真のリリース。未発火セッションでは `pathOk` も AND して弱い振りでの早期リリースを防止。
- ・タイムアウト：`（now - sArmedAtMs）≥ beatArmedTimeoutMs（800ms）`。`path/refractory` を満たしていれば拍として採用してから Idle に戻す（取りこぼし防止）。

方向非依存性の確保（ADR-0007 成果物範囲 #4）：判定は加速度ベクトルのノルムで行うため、振る方向（縦・横・斜め）に依存しない。ただし起動時 Calibrating の 2 秒間に静止していないと静止ノルム（`gravityMag`）推定がずれるため、使用者には「起動後 2 秒間は腕を自然に下ろす」だけを案内する。

2.4.2.5 OrcSenderModule の出力フェーズ

`updateOutput()` は出力フェーズで毎周期実行される。`Udp.send()` は呼ばず、EMA の「モジュール間直接呼び禁止」に従い `SystemData` 経由で `OrcNetModule` に送信を依頼する。

1. `masterNow = millis()` を取得。指揮者は自分の `millis()` がそのままマスタ時刻。
2. **BEAT 送信予約（イベント駆動）**：`data.beat.event` が立っていれば、`BeatPacket` を組み立てる。`header.seq` は `data.sender.beatSeq` をインクリメント。`header.timestampMs = masterNow`。`beatNo = data.beat.beatNo`。`playAtMasterMs = masterNow + beatLookaheadMs`。その上で `data.orcNet.pendingBeat` に格納、`pendingBeatRedundancy = beatRedundancy`、`hasPendingBeat = true` を立てる。実送信は同フェーズ後段の `OrcNetModule.updateOutput()` が担当。
3. **CTRL 送信予約（周期駆動）**：内部 `ctrlTimer` の経過 ms が `ctrlIntervalMs`（既定 50ms = 20 Hz）以上ならタイマをリセットし、`CtrlPacket` を組み立てる。`header.seq` は `data.sender.ctrlSeq` をインクリメント。`header.timestampMs = masterNow`。`bpmQ8 = bpm × 8`。`velocity = data.tempo.velocity`。`state = data.conductor.state`。`data.orcNet.pendingCtrl` に格納し `hasPendingCtrl = true` を立てる。この `timestampMs` が楽器側の時計同期サンプルとして使われる。

2.4.3 楽器ノード (node_02-05)

4 台は同一コードで動かし、差分は ProjectConfig.h と score_data.h (および score_data.cpp) にのみ閉じ込める。

2.4.3.1 モジュール構成

- ・ 入力配列: OrcNetModule, OrcReceiverModule
- ・ 出力配列: NoteSenderModule, StatusLedModule, OrcNetModule
- ・ ロジック: applyPattern() (時計同期更新・楽譜進行・細分音符予約・発音フラグ)

2.4.3.2 ProjectConfig (抜粋)

node_02/include/ProjectConfig.h の例。node_03-05 は partId と startBeatNo, および NOTE_SENDER_CONFIG.partId のみ差し替える。

ORC_NET_CONFIG (楽器)

キー	値	役割
mode	Sta	node_01 SoftAP に接続
ssid	"OrchestraAP"	接続先 SSID (指揮者と同一)
pass	"orchestra2026"	WPA2-PSK
multicastIp	239.0.0.1	指揮者と同一グループ
udpPort	5001	共通ポート

ORC_RECEIVER_CONFIG

キー	値	役割
partId	0x02	node_02 = 金管 1 (他ノードは 0x03-0x05)
startBeatNo	0	輪唱の入り拍. Version 0.2 のテスト実装は 4 ノード全て 0 で同時発音を確認. 本番の輪唱 (4 / 8 拍ずらし) は次フェーズで実装
clockSyncEmaAlpha	0.10	時計オフセット EMA 係数
clockSyncMinSamples	5	時計同期の収束判定. Playing 遷移条件には使わない (鳴らない症状の温床になるため Version 0.2 で外した). 診断ログとデバッグ表示にのみ使用
loopIntervalMs	5	ループ周期. main.cpp 側で間引きして UDP 受信遅延を抑える

Version 0.1 からの変更: beatTimeoutMs と expiredGraceMs を撤廃した. SelfRun 自走を撤廃したので前者は不要. 「過去 BEAT 切り捨て」も「鳴らない」要因になるため, 0.2 では playAtMasterMs がすでに過ぎていても **即発火 (捨てない)** 方針に倒した.

NOTE_SENDER_CONFIG

キー	値	役割
baudRate	115200	自パート Mac の USB CDC ボーレート
partId	0x02	NOTE ペイロード内識別子. 他ノードでは 0x03-0x05

STATUS_LED_CONFIG

キー	値	役割
pin	LED_BUILTIN	表示ピン
blinkIntervalMs	500	既定点滅周期

2.4.3.3 SystemData (抜粋)

楽器ノード固有のロジック中間状態として SyncLogicData (時計同期) と ReceiverLogicData (保留 BEAT キュー) を集約する.

PerformerState (列挙, Version 0.2 で SelfRun を撤廃した 3 状態)

値	意味
Idle	起動直後, SoftAP 未接続
WaitStart	SoftAP 接続済, 最初の BEAT 待ち
Playing	通常演奏, 受信 BEAT に追従して楽譜 currentEventIndex を進める, BEAT が長く来な くても Playing に留まり次 BEAT で再開

SyncLogicData (時計同期)

フィールド	役割
offsetMs	自時計 → マスタ時刻の補正 (int32_t), masterNow = millis() + offsetMs
sampleCount	EMA に積んだサンプル数
converged	sampleCount ≥ clockSyncMinSamples で true, Version 0.2 では状態遷移条件に使わず , 診断ログ/デバッ グ表示専用 (収束待ちで Playing に入れず「鳴らない」症 状の温床になったため外した)

CtrlData (直近に受信した CTRL の取り込み結果, OrcReceiverModule が書く)

フィールド	役割
bpm	bpmQ8 / 8, 細分音符オフセットや durationMs 換算に使う 参考値
velocity	CTRL の velocity (ストレッチ未実装時は 64 固定), 楽 譜 velocity と乗算合成
state	指揮者の ConductorState (0-3), デバッグ表示用
lastReceivedMs	直近 CTRL を受理した自時計時刻

PendingBeat (マスタ時刻待ち BEAT)

フィールド	役割
valid	有効エントリか
beatNo	拍番号 (重複排除キー)
playAtMasterMs	発音指定マスタ時刻
enqueuedAtMs	受信時の自時計時刻. デバッグ/遅延計測用

ReceiverLogicData (受信ロジック中間状態)

フィールド	役割
hasFirstBeat	起動以降に最初の BEAT を受理したか. WaitStart → Playing 遷移条件
lastBeatNo	直前に受理した拍番号. これと同じ beatNo は重複排除
lastBeatMs	直前に受理した自時計時刻. 診断ログ用
pending	マスタ時刻待ち BEAT (PendingBeat). 発火後は valid = false で消化

ScoreProgressData (楽譜進行)

フィールド	役割
currentEventIndex	kScore[] 配列の進行位置. BEAT 1 個ごとに 1 進める. 末尾到達で 0 に戻して曲をループ再生
pendingSub	細分音符 (8 分音符等) の予約スロット有効フラグ
pendingSubAtMs	細分音符を発火する自時計時刻 (拍頭 + subOffsetQ8 換算)
pendingSubNote / Velocity / DurationMs	細分音符の発音情報. ScoreEvent.subNote に応じて fireScoreEvent が積む

SystemData (楽器ノードの集約データ)

メンバ	役割
orcNet	OrcNetModule の受信結果 (lastCtrl/lastBeat) と送信予約 (楽器側は受信のみ使用)
led	StatusLedModule への指示値
receiver	受信ロジック中間状態 (ReceiverLogicData:hasFirstBeat・pending など)
noteOut	NoteSenderModule への発音指示 (NoteOutData)
noteSender	NOTE 送信側の内部状態 (NoteSenderData)
sync	時計同期 (SyncLogicData)
ctrl	直近受信 CTRL の取り込み結果 (CtrlData)
performer	演奏状態 (PerformerStateData:現在の PerformerState)
score	楽譜進行 (ScoreProgressData)

2.4.3.4 楽譜データ形式

楽譜は C 配列としてヘッダに埋め込み, SD カード等を使わない. score_data.h に ScoreEvent の宣言と楽譜本体 (kScore[]) の前方宣言を置き, 本体は同名の score_data.cpp に並べる.

ScoreEvent (楽譜 1 イベント, Version 0.2)

フィールド	役割
beatAt	参考値. 現在は currentEventIndex 駆動なので発火条件には使わず, 「拍番号と楽譜行を一致させて読みやすくする」目的でのみ書く (1, 2, 3, ... と並べる)
noteNumber	MIDI ノート番号. 0 なら休符
velocity	個別 velocity. CTRL の velocity と乗算合成
durationQ8	拍の 1/256 単位の発音長 (256 = 1 拍). Processing 側はこの値を ms に変換した durationMs で自動消音
flags	bit0 = NoteOn / bit1 = NoteOff (拡張用) / bit2 = 休符
subNote	細分音符 (拍頭から subOffsetQ8 拍ずれて鳴る 2 音目). 0 なら細分音符なし. 8 分音符・付点 8 分などの表現に使う
subVelocity	細分音符の velocity
subOffsetQ8	拍頭からのオフセット (128 = 半拍 = 8 分音符)
subDurationQ8	細分音符の発音長

楽譜配列の公開シンボル	kScore[]	ScoreEvent の配列. 時系列順に並べる. 末尾まで進むと currentEventIndex = 0 に戻ってループ再生
	kScoreLength	kScore[] の要素数

NoteOff の扱い (Version 0.2 の変更): 0.1 では durationQ8 経過時点で applyPattern() が pendingOff を立てて NoteOff パケットを送る方式だったが, 0.2 では **Processing 側が NotePacket.durationMs を読み取って自動消音** する方式に変更した. node_02-05 は NoteOff パケットを送らず, flags bit1 もデフォルト未使用. これに伴い noteIsSounding / noteOffAtMs 等の追跡フィールドは SystemData から削除した.

2.4.3.5 applyPattern() の処理フロー (Version 0.2)

1. **予約細分音符の発火判定**: score.pendingSub が立っており $\text{now} \geq \text{pendingSubAtMs}$ なら noteOut.pendingOn を立てて pendingSub をクリア.
2. **時計オフセット更新**: 受信フェーズで data.orcNet.hasNewCtrl OR hasNewBeat が立っていれば, $\text{sample} = \text{pkt.header.timestampMs} - \text{millis}()$ を EMA (clockSyncEmaAlpha) で data.sync.offsetMs に積む. 収束判定 ($\text{sampleCount} \geq \text{clockSyncMinSamples}$) はデバッグ表示用のフラグに留め, 状態遷移条件には使わない.

3. **演奏状態遷移 (3 状態)** : Idle → WaitStart (WiFi 接続完了) → Playing (最初の BEAT を受信, receiver.hasFirstBeat が立った). Version 0.2 で **SelfRun** を撤廃.
4. **マスタ時刻発音判定**: state == Playing かつ receiver.pending.valid なら, targetLocalMs = pending.playAtMasterMs - sync.offsetMs を計算. waitMs = targetLocalMs - now が
 - ・ ≤ 0 (既に到来 or 受信遅延) : **即発火**. pending.valid = false で消化. 受信遅延が大きくても捨てない (捨てると鳴らない症状の原因になるため).
 - ・ > 0 : 何もしない. 次ループで再評価し時刻到来したら発火.
5. **楽譜進行 (純粹 index 駆動)** : 発火イベントが立っていれば kScore[currentEventIndex] を発火し currentEventIndex を 1 進める. 末尾到達で 0 に戻してループ再生. beatAt は参考値で, 発火条件には使わない. kScore[idx].subNote != 0 なら score.pendingSub に細分音符の予約を積む (次回以降のループで step 1 が消化).
6. **velocity 合成**: 楽譜 velocity と CTRL velocity を $\lfloor (\text{score} \times \text{ctrl}) / 127 \rfloor$ で乗算合成. ストレッチ未実装時は CTRL velocity が固定 64.
7. **LED 反映**: performer.state に応じて led 出力を更新.

1 ループ 1 NoteOn の制約 (既知の制約) : noteOut は 1 スロットしか持たないため, 細分音符 (step 1) と 4 分音符 (step 5) が偶然同一ループに重なると後勝ちで上書きされる. 通常の BPM・受信ジッタの範囲では発生しないが, 本番までに「noteOut を 2 スロット化」または「同一ループに重なったら細分音符を次ループへ繰り越す」のいずれかで安全側に倒す予定.

loop() は loopIntervalMs (5 ms) で間引き, マスタ時刻判定の粒度を楽器間で揃える.

NOTE 送出は出力フェーズで NoteSenderModule が担当する. data.noteOut.pendingOn を見て 20 バイトの NotePacket (gate = 1, durationMs) を組み立て, Serial.write() の直後に Serial.flush() を呼んで USB CDC の内部バッファを即吐く (バッファに溜まると発音がバースト化するため). NoteOff は送らず, Processing 側が durationMs 経過時点で自動消音する.

2.4.3.6 ノード間差分

4 台の楽器は ProjectConfig.h と score_data.h 以外は同一コード (表 2.5).

表 2.5: 楽器ノード間差分 (本番想定 / Version 0.2 テスト実装)

ノード	partId	startBeatNo	score_data (先頭 7 音)
本番想定 (最終納品)			
node_02	0x02 (金管 1)	0	金管 1 の楽譜 (先頭から)

ノード	partId	startBeatNo	score_data (先頭 7 音)
node_03	0x03 (金管 2)	4	金管 2 (輪唱で 4 拍遅れて入る)
node_04	0x04 (金管 3)	8	金管 3 (輪唱で 8 拍遅れて入る)
node_05	0x05 (ドラム)	0	ドラム (先頭からリズム伴奏)
Version 0.2 テスト実装 (同期検証用の和音構成, <i>startBeatNo</i> は全 0 で同時発音)			
node_02	0x02 (金管 1)	0	C4 D4 E4 F4 E4 D4 C4 (ドレミファ ミレド)
node_03	0x03 (金管 2)	0	E4 F4 G4 A4 G4 F4 E4 (3 度上)
node_04	0x04 (木管 1)	0	G4 A4 B4 C5 B4 A4 G4 (5 度上, 木 管音域試行)
node_05	—	—	未実装 (本番で実装)

テスト実装では「3 台同時に同じ拍を踏むか」を MOP-1 検証用に試したかったため *startBeatNo* を全 0 で揃え、3 度・5 度に積んで C major 圏内のハモリが出る配置にした。本番の輪唱 (4 / 8 拍ずらし) は *startBeatNo* を表のとおり差し替えるだけで切り替わる。beatAt は単調増加の整数を入れておくが Version 0.2 では発火条件として参照しない (*currentEventIndex* 駆動)。

2.5 テストの設計

§1.1 で定義した MOP を実機で測るための試験計画と、要求 → 検証の対応関係を示す。

2.5.1 試験戦略 (単体 → 結合 → システム)

PlatformIO の test/ を使い、共通層と *applyPattern()* を中心に単体テストを整備する。EMA に従い判断ロジックは *IModule* 派生でなく *applyPattern()* 関数として書くため、テストでは *SystemData* をテスト側で組み立てて *applyPattern()* を呼ぶだけでロジック単体を検証できる。

表 2.6: 単体テスト対象

対象	確認内容	手段
ModuleTimer	<i>setTime()</i> 後の経過時間判定	<i>millis()</i> モック差替え
OrcProtocol シリアライズ	CTRL/BEAT/NOTE のラウンドトリップ	バイト列比較
<i>applyPattern()</i> 拍検出	記録 IMU 時系列で拍位置一致	ゴールデンテスト

対象	確認内容	手段
applyPattern() テンポ推定	一定間隔の拍で BPM 収束	単調列テスト
applyPattern() 楽譜進行	lastBeatNo 増加で正しい順序で pendingOn	状態機械テスト
applyPattern() 細分音符予約	ScoreEvent.subNote != 0 で pendingSub が積まれ, pendingSubAtMs 到達で発火	状態機械テスト
applyPattern() 時計同期	timestampMs 系列に対し offsetMs が EMA 収束	数値列テスト
applyPattern() 発音判定	playAtMasterMs に対し未来 / 過去（即発火）の 2 分岐	状態機械テスト

結合テストは Tier 単位で段階的に積み上げる。

表 2.7: 結合テスト Tier

Tier	構成	確認事項	機材
T1	node_01 単体	シリアルに beatNo/bpm が出る. 手で振って拍が出る	PC + USB
T2	node_01 + node_02 node_02	+ node_02 が BEAT 受信 → NOTE 出力	受信スクリプト
T3	node_01 node_02-05	+ 4 台がほぼ同時に NOTE. 同期誤差 ≤ 20 ms (MOP-1)	USB ハブ + ログ集約
T4	全ノード + PC Processing	実音再生. 指揮速度を変えると BPM が追従	PC + スピーカ + Processing

2.5.2 試験項目 (TPM)

各 MOP に対する試験項目を表 2.8 に示す。各試験の「目標」「合否基準」欄の数値は、すべて §1.1.3（各 MOP 目標値の根拠）で示した合奏知覚閾値・DTM 推奨レイテンシ・指揮ジェスチャ認識の先行研究・無線 LAN のパケットロス品質基準などに基づいており、TPM 側で独立に値を定めているわけではない（出典は MOP 表側に集約）。

表 2.8: 試験項目 (TPM)

ID	項目	目標	手順	合否基準	時期
TP-1	拍検出精度	MOP-3: $\geq 90\%$	80/120/160 BPM のメトロノームに合わせて指揮し、30 秒の検出 vs 実拍数	$\geq 90\%$	W6 (M3)
TP-2	通信遅延	MOP-2: $\leq 10\text{ ms}$	node_01 が BEAT を送信した直後に同 BEAT を自身でループバック受信し、送信～受信の時刻差を node_01 単体の millis() で 100 サンプル計測 (時計同期不要)	平均 $\leq 10\text{ ms}$	W7 (M4)
TP-3	同期誤差	MOP-1: $\leq 20\text{ ms}$	4 楽器が記録した 発音時のマスタ時刻 (masterNow at NOTE 送出時) を比較、100 拍ぶんログ取得	最大差 $\leq 20\text{ ms}$ (マスタクロック方式により実機では 5 ms 以下を目標)	W9 (M5)
TP-4	テンポ追従	MOP-4: $\leq 2\text{ 拍}$	80 $\rightarrow$ 120 BPM 切替時に楽器側 BPM が 120 に入るまでの拍数	$\leq 2\text{ 拍}$	W9 (M5)
TP-5	パケロス耐性	MOP-5	ルータで擬似 5% パケロスを設定、演奏破綻なきこと	聴感破綻なし	W11 (M7)
TP-6	CPU 負荷	MOP-6: $\leq 2\text{ ms}$	micros() で入力フェーズ所要時間を 10 分ログ取得	最大 $\leq 2\text{ ms}$	W11 (M7)
TP-7	起動時間	MOP-7: $\leq 5\text{ s}$	電源投入 $\rightarrow$ 最初の CTRL 送出までを 5 回平均	$\leq 5\text{ s}$	W11 (M7)
TP-8	強弱制御 (ストレッチ)	MOP-8	小・中・大の振幅で velocity 出力値を記録	3 段階分離	W10 (M6)

2.5.3 要求 $\rightarrow$ 検証の対応

表 2.9: 要求と検証の対応表

要求 ID	要求内容	検証方法	関連 MOP
R-1	5 台同期演奏	T4 で聴感確認 + MOP-1/2 実測	MOP-1, MOP-2
R-2	輪唱曲（主旋律 3 + リズム 4 パート楽譜を用意し 1 以上）	T4 で演奏	—
R-3	可変テンポ追従	指揮 BPM 階段変化で実測	MOP-4
R-4 (S)	強弱制御	指揮振幅変化で	MOP-8
R-Internal-1	通信の信頼性	MOP-5（5% パケロスで継続）	MOP-5
R-Internal-2	CPU リソース	MOP-6（入力 ≤ 2 ms）	MOP-6
R-Internal-3	起動時間	MOP-7（5 s 以内）	MOP-7

2.5.4 テスト実装で確定した値（Version 0.2 追記）

firmware/test/ 一式（指揮者ノード node_01 + 楽器 3 台 node_02-04 + Mac 3 台 Processing）で動作確認を行い、0.1 時点の暫定値・暫定方針のうち以下を確定した（表 2.10）。未着手の項目は引き続き §2.5.5 に残す。

表 2.10: テスト実装で確定した値・方針

項目	確定値・方針	確定の根拠
指揮者 MCU	XIAO ESP32-S3 Sense (変更なし)	UNO R4 WiFi の WiFiS3 は AT bridge 経由で SoftAP 動作にコミュニティ報告レベルの不安があり、0.1 時点で既に置き換えを採択。テスト実装でも安定動作を確認
指揮者 IMU	MPU6050 / GY-521 モジュール	ピンヘッダ・LDO・プルアップ実装済のブレイクアウトで結線が {3V3, GND, SDA, SCL, AD0} の 5 本だけで完結。Wire で直接レジスタ叩きで動作
SoftAP マルチキャスト到達性	239.0.0.1:5001 で確定	ESP32-S3 Arduino-ESP32 の WiFi.softAP() + beginMulticast() で問題なく動作。サブネットブロードキャストへのフォールバックは不要となった

項目	確定値・方針	確定の根拠
拍検出方式	動加速度ノルム + 経路長 ゲート + デバウンスリ リース	「ノルム閾値 + 不応期」のみだとピーク 後発火・連続スイング誤検出が頻発した ため、経路長 0.20m 早期発火 → ピー ク 40% 切りでリリースの構成に作り直 した (§2.4.2)
拍検出閾値	beatDynThresholdG = 1.20	重力補正後の動加速度ノルムで判定、自 然な指揮ストロークのピーク 1.5-3.0g から確実に Armed に入り、かつ軽い揺 れで誤発火しない値として収束
不応期	beatRefractoryMs = 350ms	1 振り中の振り戻しを 2 拍として拾う 症状の対策、約 170 BPM 上限相当
楽譜進行方式	純粹 currentEventIndex 駆動 + 末尾ループ	「effectiveBeatNo - startBeatNo」と beatAt の照合方式は 1 拍取りこぼすと 曲頭まで戻る挙動が予測しづらく、 「BEAT 1 個 = index 1 進める」に簡素化
NoteOff の扱い	Processing 側で durationMs から自動消音	node_02-05 から NoteOff パケットを 送る方式は USB CDC のバッファリン グ・順序待ちで「ノートが鳴りっぱなし」 になりやすく、Mac 側で自動消音 する方式に倒した
楽器側 Playing 遷移	最初の BEAT 受信のみ (時計同期収束待ちなし)	CTRL 数発の収束待ちは「鳴らない」温 床、最初の数拍のズレを許容して offset は EMA で自然収束させる運用 に切替
SelfRun 自走	撤廃	「BEAT が来ない → 仮想 BEAT で内挿」 が予期せず発火し続ける症状（指揮者再 起動中に楽器だけ走る等）を排除、 BEAT が長く来なくても Playing に留ま り次 BEAT で再開する
過去 BEAT の扱い	即発火（捨てない）	expiredGraceMs 超過でスキップする方 式は「鳴らない」要因になったため撤 廃、連発冗長性で実用上は問題なし

項目	確定値・方針	確定の根拠
NoteSender flush	Serial.flush() を毎パケット送出後に呼ぶ	USB CDC が小さい書込みを内部バッファで束ねるため、flush なしでは発音が「ためてバースト」状態になる症状を確認
PC 側ポリフォニー	MAX_POLYPHONY = 16	4 楽器 × 同時数音まで余裕を持って鳴らせ、超過時は最古の active を強制 release してリソース確保。サイン波 + 線形 ADSR のシンプル合成で十分動作
PC 側スレッド	ConcurrentLinkedQueue で serialEvent 隔離	Processing の serialEvent は別スレッドで呼ばれ、Voice 操作で ConcurrentModificationException を踏むため、受信パケットをキューに積み draw() スレッドで消化する構成に整理
SERIAL_DEBUG ビルドフラグ	platformio.ini で 0/1 切替	0 でバイナリ NotePacket 送信, 1 でテキストログ送信。Processing 連携と pio device monitor を排他で切替。ARM_END サマリログで拍検出のチューニングが追える

2.5.5 実機検証で確定する未確定事項

§2.5.4 に移した項目以外で、引き続き実機 T1-T4 結合試験 (§2.5) で確定する項目を表 2.11 に列挙する。

表 2.11: 実機で確定する仮値

項目	仮値	確定の見極め
BEAT 連発回数	beatRedundancy = 2	MOP-1 評価で「2 で十分」を確認後に確定。T3 のパケロス実測値を見て 1-3 連発に調整
lookahead	beatLookaheadMs = 50	T3 で振り → 音の体感遅延と最遠スレーブの waitMs ≤ 0 比率を見て 30-80 ms で調整。人間の視聴覚同時知覚閾 80 ms 内に収める前提

項目	仮値	確定の見極め
時計同期 EMA 係数	clockSyncEmaAlpha = 0.10	T2 で offsetMs の収束時間と定常ジッタを見て 0.05-0.20 で調整
WiFi チャンネル	channel = 6	会場の電波状況を Site Survey して空きチャンネルへ
LED 色・点滅周期	1/2/5 Hz の点滅	視認性確認後に決定
強弱推定（ストレッチ）	仮: 振り幅 → velocity 線形写像の 3 段階	TP-8 にて p / mf / f が分離して聞こえる写像式に調整. Version 0.2 時点では CTRL.velocity = 64 固定でスタブ
ビブラート（ストレッチ）	仮: 角速度の高周波成分から振幅変調 0-20 Hz	必達機能完了後に着手. Processing 側でフィルタ / LFO 適用
ドラム node_05	仮: 金管同様の ScoreEvent で実装	楽譜とパーカッション音色の合意後に実装. noteNumber は GM ドラムマップ準拠を想定
楽譜データスキーマ	0.2 時点 ScoreEvent で確定	齋藤担当との {pitch, durationBeats} 形式との橋渡しは「楽譜エディタ → ScoreEvent 配列を生成」の変換層を用意して吸収
ESP-NOW（ストレッチ）	不採用（WiFi UDP マルチキャスト主案）	テスト実装でマルチキャストが安定動作したため、採否は据え置き

第 3 章 生成 AI の利用について

本書の執筆にあたっては、生成 AI として **Claude Opus (Anthropic)** を補助的に利用した。提出規約上、本計画書については生成 AI 利用申告書の提出が明示的に求められているわけではないが、透明性のため、どのように利用したかを簡潔に記しておく。

利用方法は **対話を通じた共同執筆** である。設計の方針、章立て、図表の構成、文章表現の選び方などについて Claude Opus と繰り返し議論し、提案された文案を著者が精査・修正・取舍選択しながら本書の形にまとめた。コード例や設計値（ピン配置・通信パラメータ・状態遷移など）は、著者が自ら設計・検証したものを正としており、AI による出力をそのまま採用した箇所はない。

AI の出力に対しては、著者の Embedded-Module-Architecture (EMA) の仕様および Arduino/ESP32 の実装制約と整合するかを著者自身がレビューし、事実誤認や設計と食い違う記述は都度書き換えた。本書に残る記述の妥当性および誤りの責任は、すべて著者が負う。

第 4 章 参考文献

本書で MOP 目標値の根拠として引用した日本語文献を以下に示す。URL はいずれも 2026 年 4 月時点でネット上で誰でも閲覧可能なものを選んだ。

参考文献

- [1] 河瀬諭, 「合奏における演奏者間コミュニケーション—タイミング調整とその手がかり—」, 心理学評論, 第 57 巻第 4 号, pp.495–510, 2014 年, https://www.jstage.jst.go.jp/article/sjpr/57/4/57_495/_article/-char/ja/ (合奏のタイミング知覚に関する心理学的研究のレビュー論文。演奏者間のズレが数 10 ms 程度なら知覚されず, 100 ms を超えるとズレと判断される旨を, Rasch (1979), Keller & Appel (2010) などの先行研究を引いて整理)。
- [2] 情報処理学会, 「広帯域ネットワークを用いた遠隔同時音楽演奏の実験とその考察」, 情報処理学会研究報告 / 論文誌 (J-STAGE 経由公開), https://ipsj.ixsq.nii.ac.jp/ej/?action=repository_action_common_download&item_id=34724&item_no=1&attribute_id=1&file_no=1 (広帯域ネットワーク上でのリアルタイム音楽演奏における許容遅延の実験的考察)。
- [3] 宇佐聡史, 持田康典, 「マルチモーダル指揮シミュレータ」, 日本ファジィ学会誌, 第 10 巻第 4 号, pp.707–716, 1998 年, https://www.jstage.jst.go.jp/article/jfuzzy/10/4/10_KJ00002086732/_article/-char/ja/ (加速度センサ等を用いて指揮ジェスチャを認識し, HMM とファジィ推論で拍タイミング・テンポ変動・ダイナミクスをリアルタイム認識する研究)。
- [4] 新山王政和, 「指揮基本動作における初心者と熟達者の動作タイミングの違いに関する分析的研究—拍点と打点の関係, 及び速度変化に基づく指揮基本原則の再考察—」, 音楽教育学, 第 34 巻第 2 号, pp.1–12, 2004 年, https://www.jstage.jst.go.jp/article/jjomer/34/2/34_1/_article/-char/ja/ (熟達指揮者の打点が拍点に先行すること, 加速度の上下方向比など, 拍検出・テンポ追従の設計上の手がかりとなる定性的特徴を提示)。
- [5] 情報通信技術委員会 (TTC), 「JT-Y1541 IP ベースサービスにおけるネットワーク性能目標」 (ITU-T 勧告 Y.1541 日本語標準), <https://www.ttc.or.jp/application/files/1915/5435/3364/JT-Y1541v2.pdf> (IP ネットワークの品質クラスごとに, 遅延・ジッタ・パケットロス率の目標値を規定する国際勧告の日本語標準版。最も厳し

いクラス 0 でロス率目標 1×10^{-3}).

- [6] オプテージ (eo 光), 「パケットロス (パケロス) とは? 発生原因や対策を解説」, <https://eonet.jp/column/recommend/what-is-packet-loss.html> (家庭用 IP 通信の実観測ベースで, パケットロス率の目安として 1 % 以下=良好, 3 % 超=遅延や不具合, 10 % 超=ストレスを感じる水準と整理).
- [7] DeNA Engineering Blog, 「無線 LAN の通信品質を見える化する話」, <https://engineering.dena.com/blog/2018/12/wireless-lan-quality-visualize/> (無線 LAN におけるマルチキャストの取りこぼしや有線比でのロス増加が有意に観測されるという企業エンジニアリング部門による計測事例).